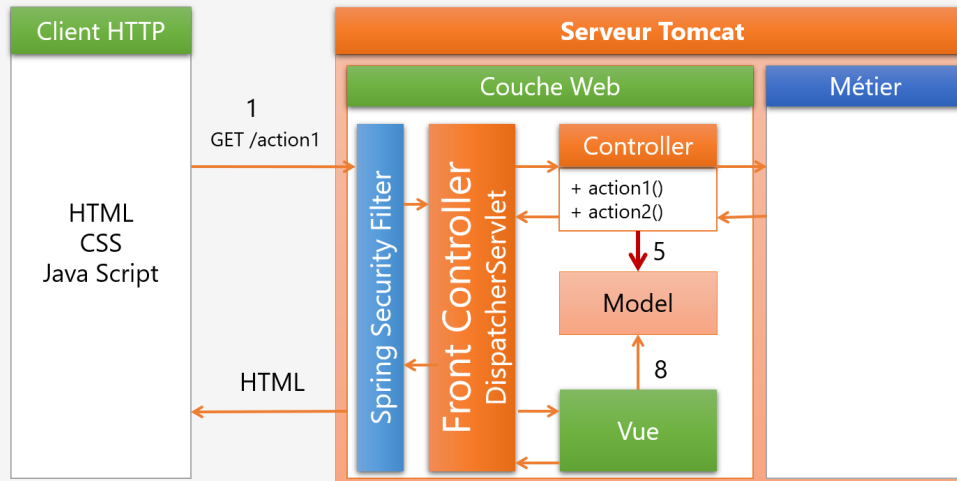


Spring Security 6



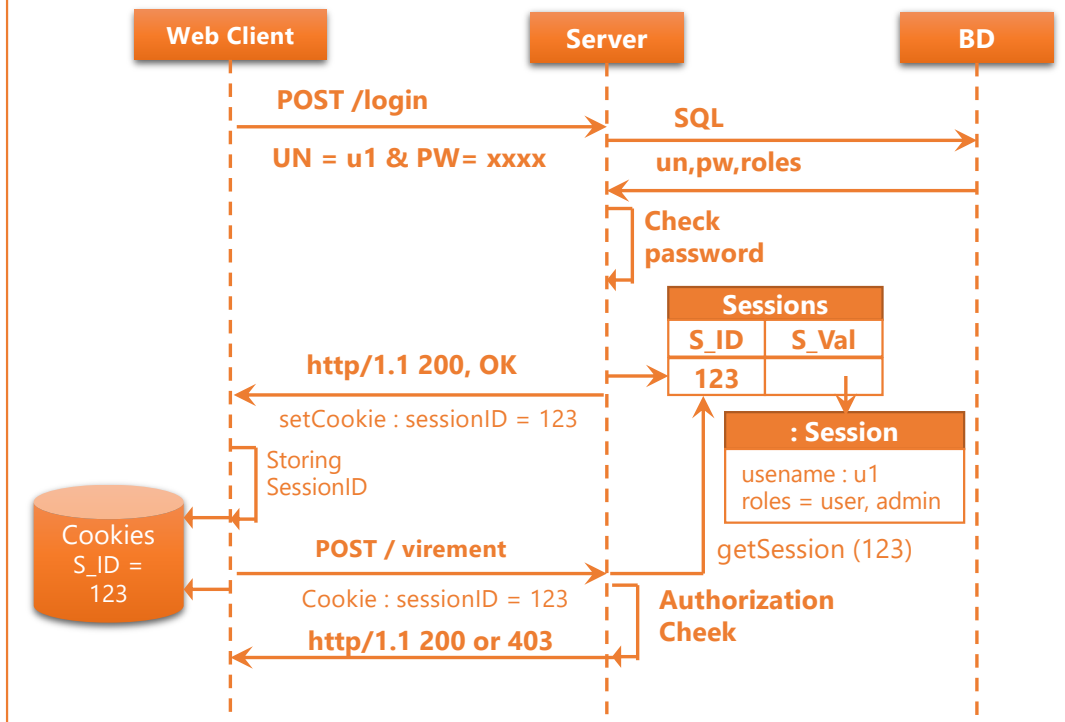
Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, UH2C, Lab. Informatique, Intelligence Artificielle et Cyber Sécurité (ZIACS)

Systèmes d'authentification

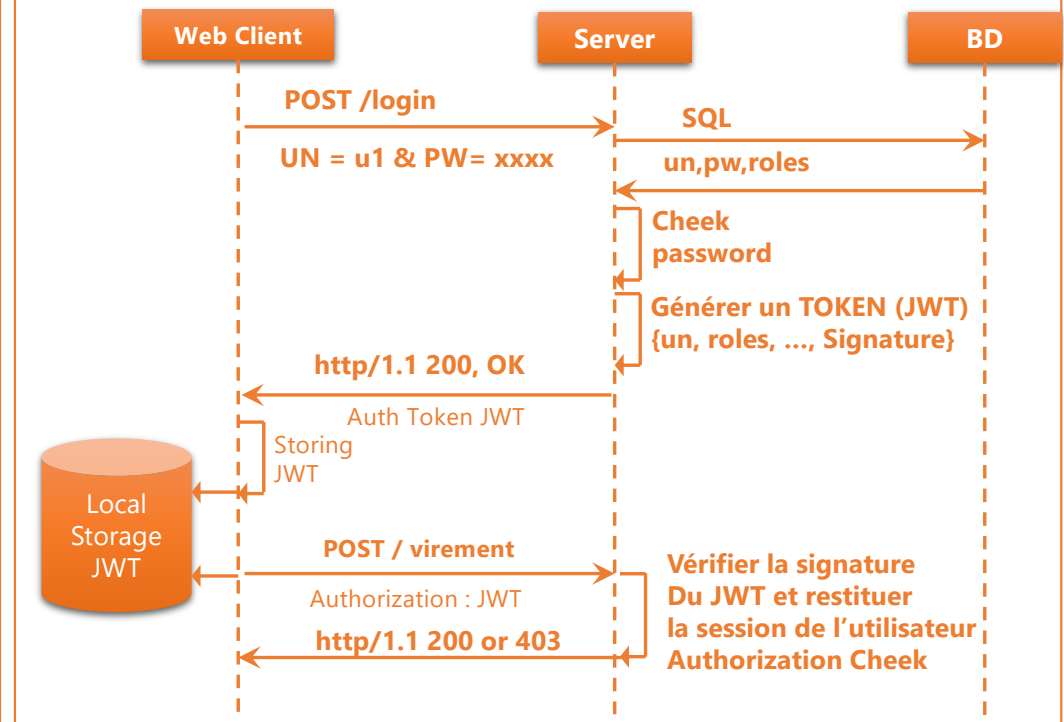
► Deux types de modèles d'authentification :

- **Statefull** : Les données de la session sont enregistrées coté serveur d'authentification
- **Stateless** : les données de la session sont enregistrées dans un jeton d'authentification délivré au client.

Authentification Statefull

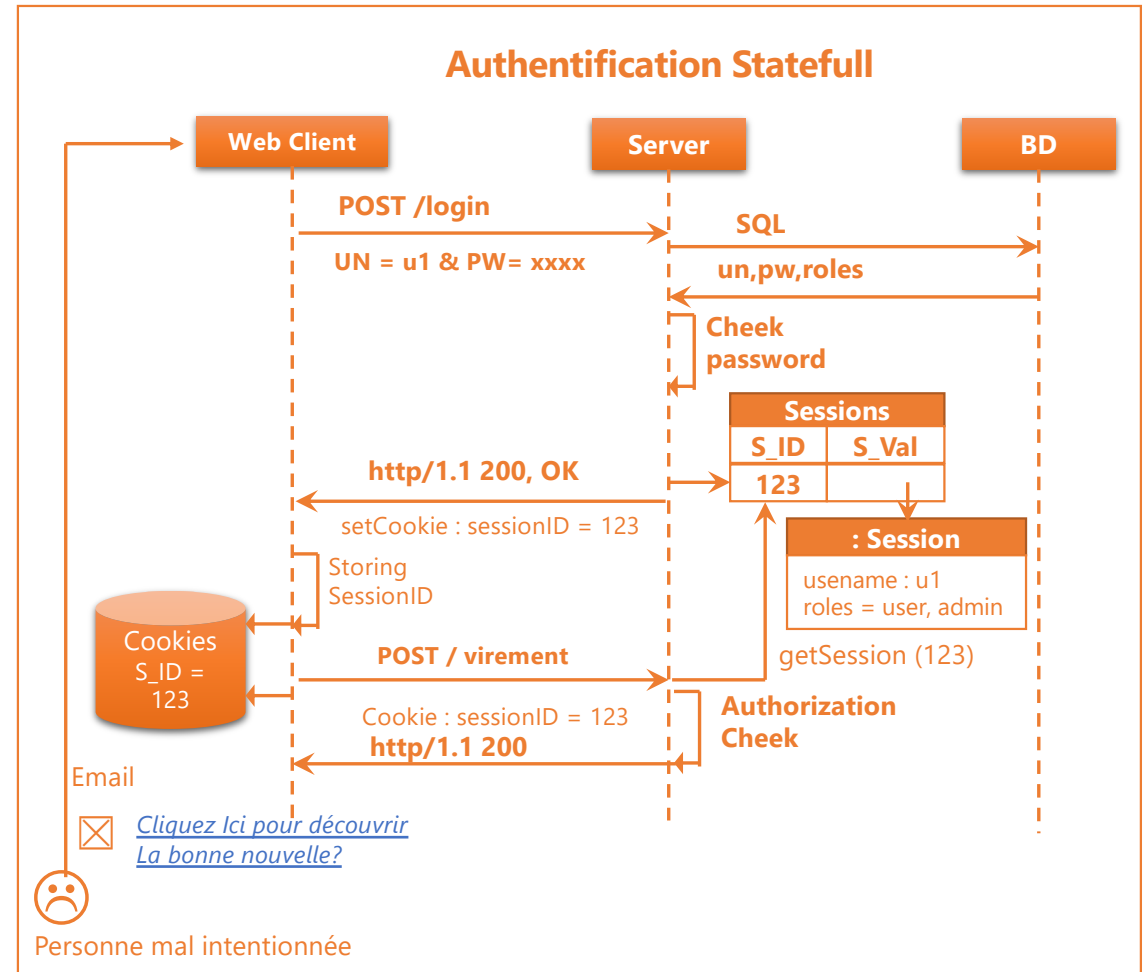


Authentification Stateless



Quelques failles de sécurité : Cross Site Request Forgery (CSRF)

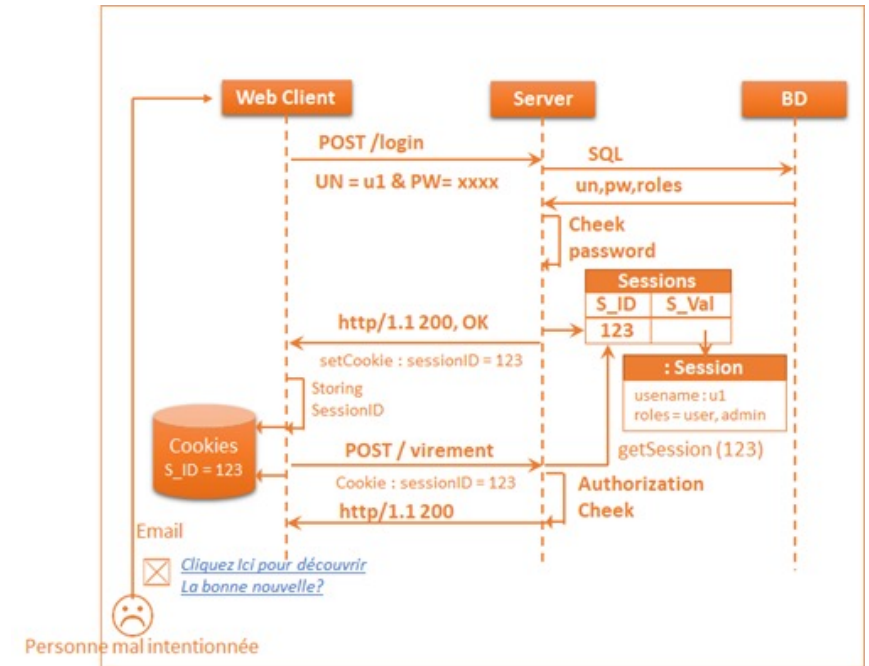
- ▶ En sécurité informatique, le Cross-Site Request Forgery, abrégé CSRF est un type de vulnérabilité des services d'authentification web.
- ▶ L'objet de cette attaque est de transmettre à un utilisateur authentifié
- ▶ Une requête HTTP falsifiée qui pointe sur une action interne au site,
- ▶ Afin qu'il l'exécute sans en avoir conscience et en utilisant ses propres droits.
- ▶ L'utilisateur devient donc complice d'une attaque sans même s'en rendre compte.
- ▶ L'attaque étant actionnée par l'utilisateur, un grand nombre de systèmes d'authentification sont contournés.



Quelques failles de sécurité : Cross Site Request Forgery (CSRF)

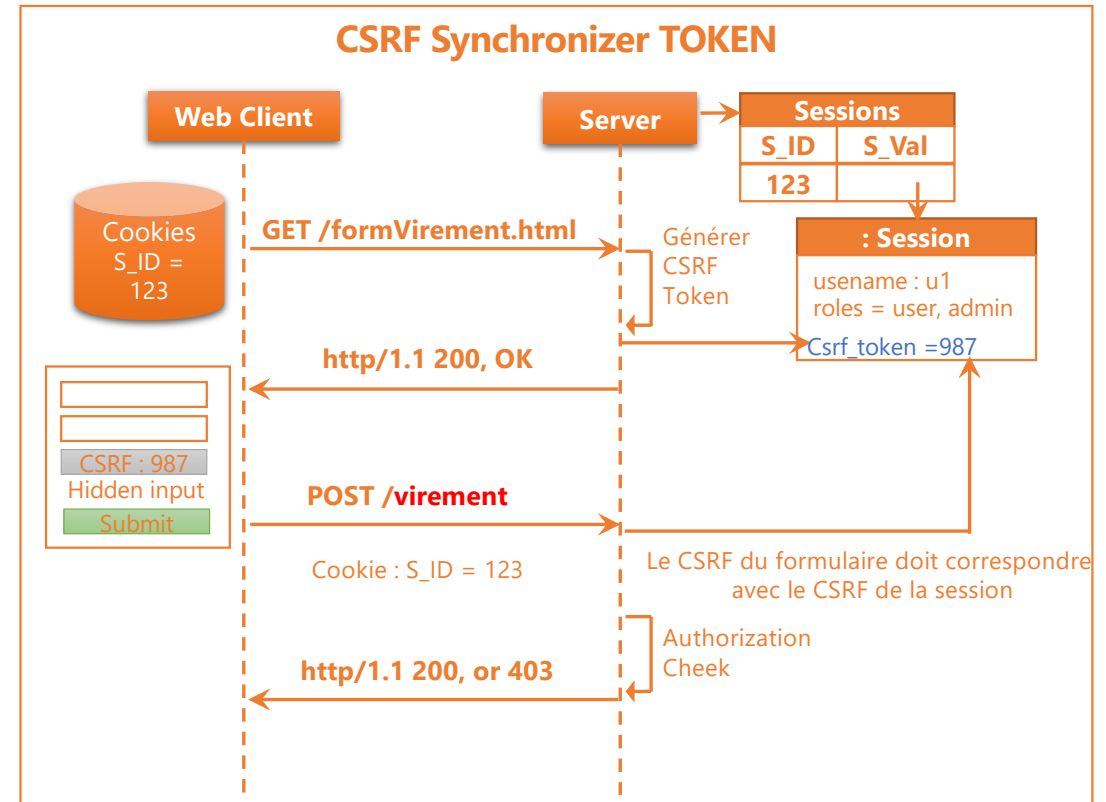
• Exemple d'attaque CSRF :

- Yassine possède un compte bancaire d'une banque qui lui donne entre autres la possibilité d'effectuer en ligne des virements de son compte vers d'autres comptes bancaires.
- L'application de la banque utilise un système d'authentification basé uniquement sur les sessions dont les SessionID sont stockées dans les cookies.
- Sanaa possède également un compte dans la même banque. Elle connaît facilement la structure du formulaire qui permet d'effectuer les virements.
- Sanaa envoie à Yassine, un email contenant un message présentant un lien hypertexte demandant à Yassine de cliquer sur ce lien pour découvrir son cadeau d'anniversaire.
- Ce message contient également un formulaire invisible permettant de soumettre les données pour effectuer un virement. Ce formulaire contient entre autres des champs cachés :
 - Le montant du virement à effectuer vers
 - Le numéro de compte de Sanaa
- Au moment où Yassine reçoit son message, la session de son compte bancaire est ouverte. Son Session ID est bien présent dans les cookies.
- En cliquant sur le lien, Yassine vient d'effectuer un virement de son compte vers celui de Sanaa, sans qu'il le sache.
- En effet, en cliquant sur le lien, les données du formulaire caché sont envoyées au bon script côté serveur, et comme les cookies sont systématiquement envoyés au serveur du même domaine, le serveur autorise cette opération car, pour lui, cette requête vient d'un utilisateur bien authentifié ayant une session valide.

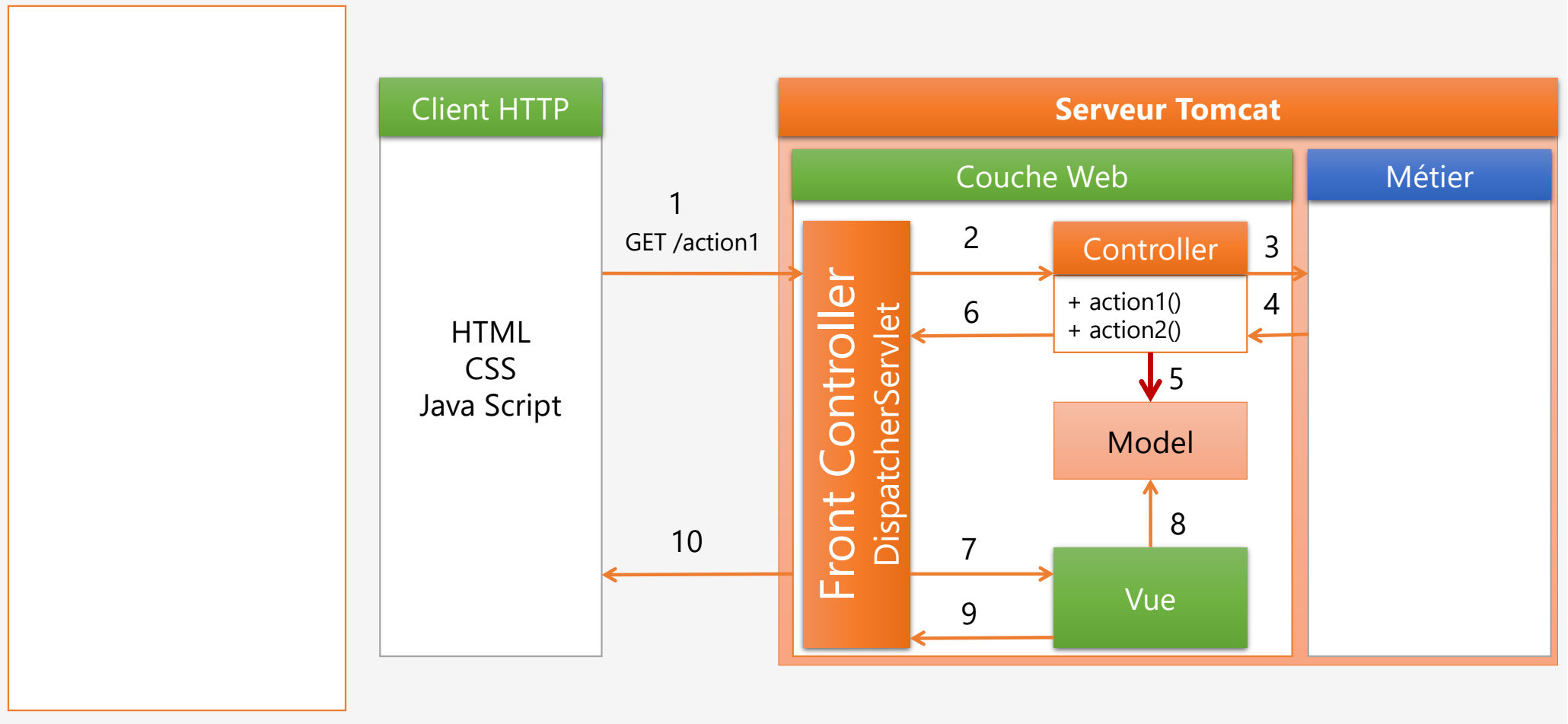


Préventions contre les attaques CSRF

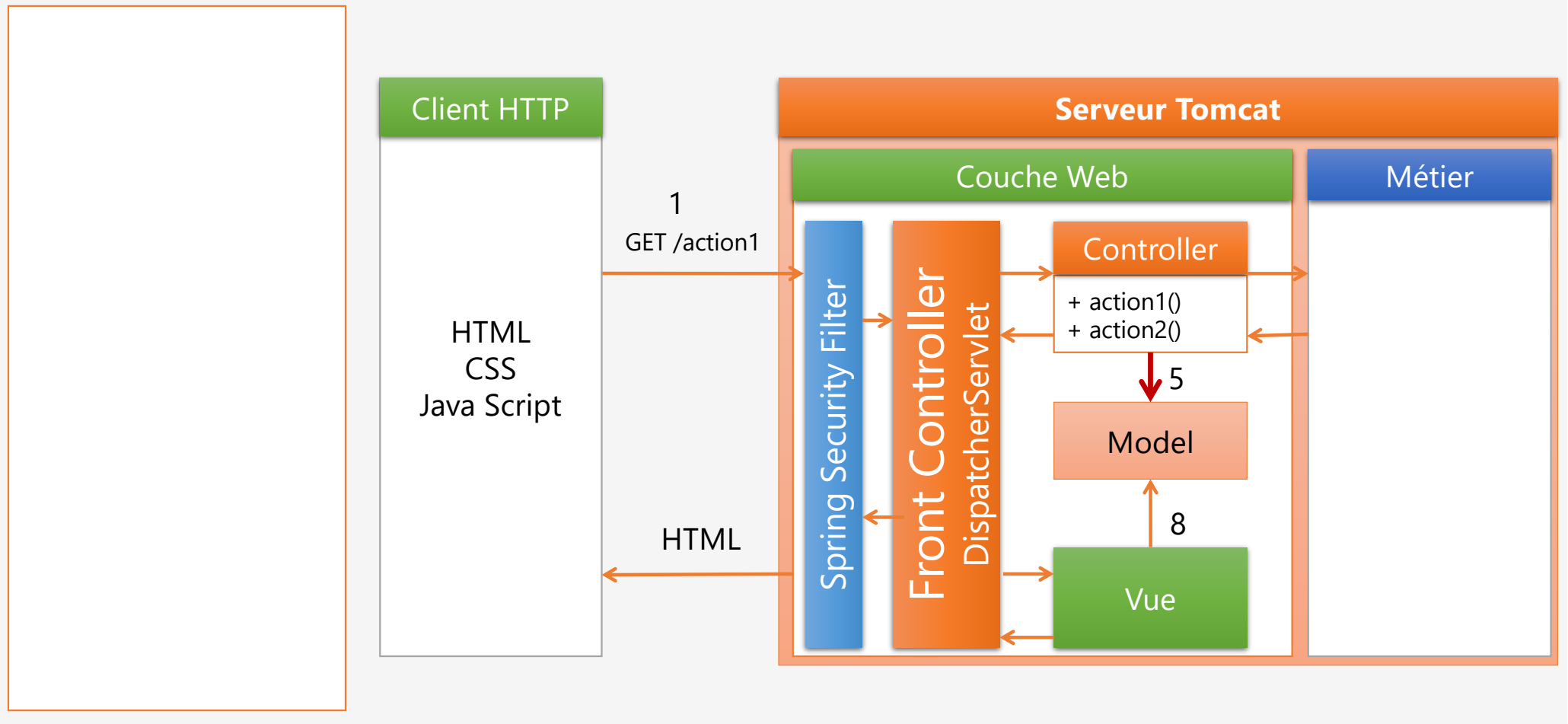
- Utiliser des jetons de validité (CSRF Synchronizer Token) dans les formulaires :
- Faire en sorte qu'un formulaire posté ne soit accepté que s'il a été produit quelques minutes auparavant. Le jeton de validité en sera la preuve.
- Le jeton de validité doit être transmis souvent en paramètre (Dans un champs de type Hidden du formulaire) et vérifié côté serveur.
- Autres présentions :
 - Éviter d'utiliser des requêtes HTTP GET pour effectuer des actions critiques de modification des données (Ajout, Mise à jour, Suppression) : cette technique va naturellement éliminer des attaques simples basées sur les liens hypertexte, mais laissera passer les attaques fondées sur JavaScript, lesquelles sont capables très simplement de lancer des requêtes HTTP POST.
 - Demander des confirmations à l'utilisateur pour les actions critiques, au risque d'alourdir l'enchaînement des formulaires.
 - Demander une confirmation de l'ancien mot de passe à l'utilisateur pour changer celui-ci ou utiliser une confirmation par email ou par SMS.
 - Effectuer une vérification du référent dans les pages sensibles : connaître la provenance du client permet de sécuriser ce genre d'attaques. Ceci consiste à bloquer la requête du client si la valeur de son référent est différente de la page d'où il doit théoriquement provenir.



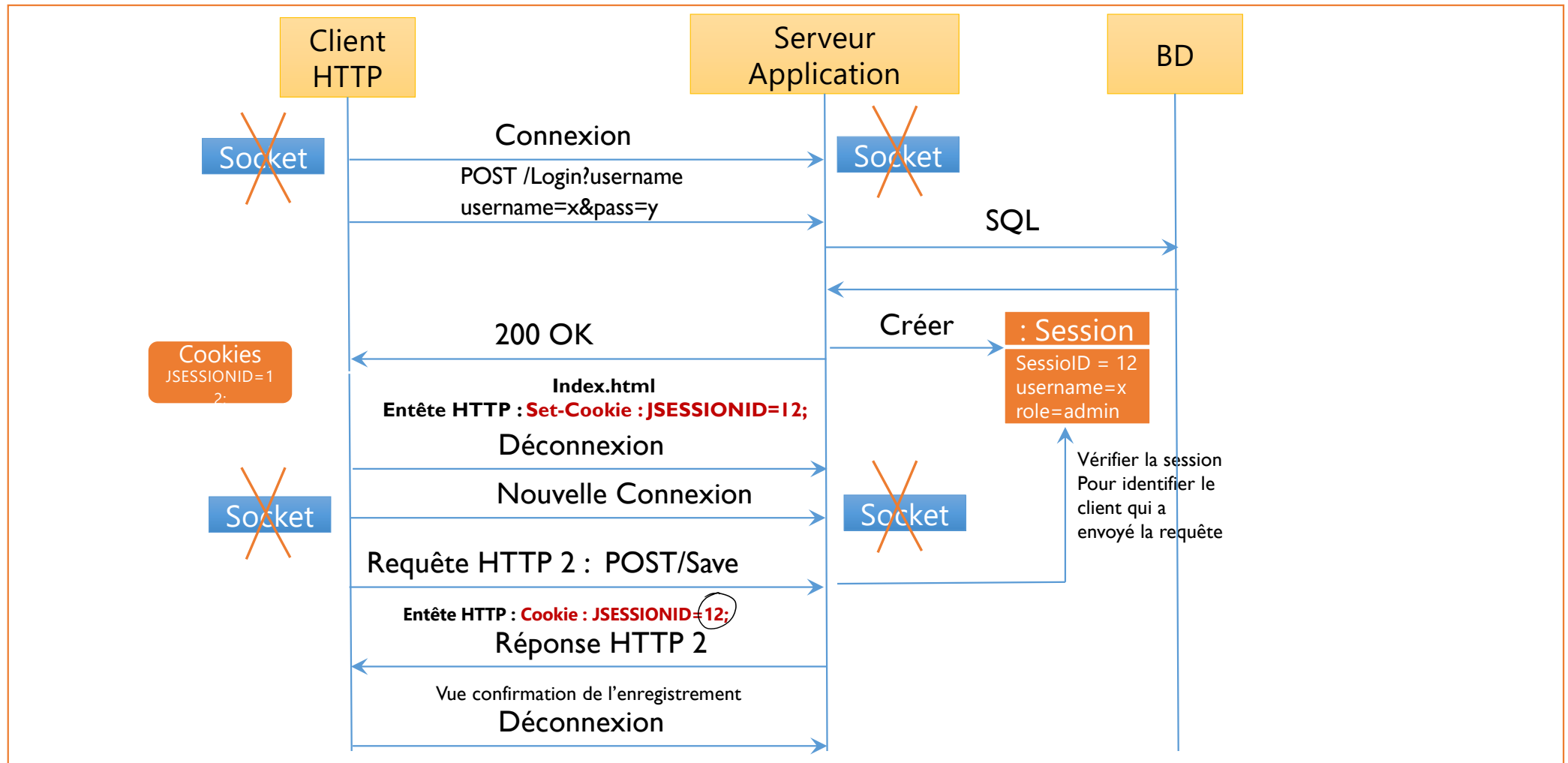
Architecture Spring MVC



Architecture Spring MVC



Authentification Statful basée sur les Sessions et Cookies



Configuration de Spring Security

- La sécurité applicative se résume à deux problèmes plus ou moins indépendants :
 - **l'authentification** (qui êtes-vous ?)
 - **et l'autorisation** (qu'êtes-vous autorisé à faire ?) ou "contrôle d'accès"
- Spring Security a une architecture conçue pour séparer l'authentification de l'autorisation et offre des stratégies extensibles pour les deux.

Configuration de Spring Security

- Spring Security est un module de Spring qui permet de sécuriser les applications Web.
- Spring Security configure des filtres (springSecurityFilterChain) qui permet d'intercepter les requêtes HTTP et de vérifier si l'utilisateur authentifié dispose des droits d'accès à la ressource demandée.
- Les actions du contrôleur ne seront invoquées que si l'utilisateur authentifié dispose de l'un des rôles attribués à l'action.
- Spring Security peut configurer les rôles associés aux utilisateurs en utilisant différentes solution (Mémoire, Base de Données SQL, LDAP, etc..)

Dépendances Maven

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-security</artifactId>  
</dependency>
```

```
<dependency>  
  <groupId>org.thymeleaf.extras</groupId>  
  <artifactId>thymeleaf-extras-springsecurity6</artifactId>  
  <version>3.1.0.M1</version>  
</dependency>
```

SecurityConfig.java : inMemoryAuthentication

@Configuration

@EnableWebSecurity

@EnableMethodSecurity(prePostEnabled = true)

public class SecurityConfig {

@Bean

public InMemoryUserDetailsManager inMemoryUserDetailsManager(PasswordEncoder passwordEncoder){

return new InMemoryUserDetailsManager(

User.withUsername("user1").password(passwordEncoder.encode("1234")).roles("USER").build(),

User.withUsername("user3").password(passwordEncoder.encode("1234")).roles("USER").build(),

User.withUsername("admin").password(passwordEncoder.encode("1234")).roles("USER", "ADMIN").build()

);

}

@Bean

public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws Exception {

httpSecurity.formLogin().loginPage("/login").defaultSuccessUrl("/", true).permitAll();

httpSecurity.csrf(csrf->csrf.ignoringRequestMatchers(new AntPathRequestMatcher("/h2-console/**")));

httpSecurity.authorizeHttpRequests().requestMatchers(new AntPathRequestMatcher("/webjars/**")).permitAll();

httpSecurity.authorizeHttpRequests().requestMatchers(new AntPathRequestMatcher("/h2-console/**")).permitAll();

httpSecurity.headers().frameOptions().sameOrigin();

httpSecurity.authorizeHttpRequests().requestMatchers("/admin/**").hasRole("ADMIN");

httpSecurity.authorizeHttpRequests().requestMatchers("/user/**").hasRole("USER");

httpSecurity.authorizeHttpRequests().anyRequest().authenticated();

httpSecurity.exceptionHandling().accessDeniedPage("/notAuthorized");

return httpSecurity.build();

}

}

@SpringBootApplication

public class HospitalAppApplication {

public static void main(String[] args) {

SpringApplication.run(HospitalAppApplication.class, args);

}

@Bean

PasswordEncoder passwordEncoder(){

return new BCryptPasswordEncoder();

}

}

SecurityConfig.java : JdbcAuthentication

Table users

username	password	enabled
admin	\$2a\$10\$FNw7298vHs5nfDVg9ew00.nT49cn.../pkSAqBosRr5...	1
user1	\$2a\$10\$t46eLP82FfQS9olmy.ua5e170WIAG5fjdZkpsvD2uvc...	1

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {
    @Bean
    public JdbcUserDetailsManager jdbcUserDetailsManager(DataSource dataSource){
        JdbcUserDetailsManager jdbcUserDetailsManager=new JdbcUserDetailsManager(dataSource);
        return jdbcUserDetailsManager;
    }
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws Exception {
        // même code
    }
}
```

Table authorities

username	authority
admin	ROLE_ADMIN
admin	ROLE_USER
user1	ROLE_USER

schema.sql

```
create table users(username varchar(50) not null primary key,password varchar(500) not null,enabled boolean not null);
create table authorities (username varchar(50) not null,authority varchar(50) not null,constraint fk_authorities_users foreign key(username) references users(username));
create unique index ix_auth_username on authorities (username,authority);
```

```
@Bean
CommandLineRunner commandLineRunner(JdbcUserDetailsManager jdbcUserDetailsManager){
    PasswordEncoder passwordEncoder=passwordEncoder();
    return args -> {
        UserDetails admin= User.withUsername("admin").password(passwordEncoder.encode("1234")).roles("USER","ADMIN").build();
        UserDetails user1=User.withUsername("user1").password(passwordEncoder.encode("1234")).roles("USER").build();
        jdbcUserDetailsManager.createUser(admin); jdbcUserDetailsManager.createUser(user1);
    };
}
```

SecurityConfig.java : JdbcAuthentication

Table users

username	password	enabled
admin	\$2a\$10\$FNw7298vHs5nfDVg9ew00.nT49cn../pkSAqBosRr5...	1
user1	\$2a\$10\$t46eLP82FfQS9olmy.ua5e170WIAG5fjdZkpsvD2uvc...	1

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {

    @Bean
    public DataSource h2DataSource(){
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.H2)
            .setName("users-db")
            .addScript(JdbcDaoImpl.DEFAULT_USER_SCHEMA_DDL_LOCATION)
            .build();
    }

    @Bean
    public JdbcUserDetailsManager jdbcUserDetailsManager(DataSource dataSource){
        JdbcUserDetailsManager jdbcUserDetailsManager=new JdbcUserDetailsManager(dataSource);
        return jdbcUserDetailsManager;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws Exception {
        // même code
    }
}
```

Table authorities

username	authority
admin	ROLE_ADMIN
admin	ROLE_USER
user1	ROLE_USER

UserDetailsService Authentication

```
@Service
@Slf4j @AllArgsConstructor
public class UserDetailsServiceImpl implements UserDetailsService {
    private AccountService accountService;
    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        AppUser appUser=accountService.loadUser(username);
        if(appUser==null) throw new UsernameNotFoundException("User Not found "+username);
        List<AppRole> appRoles = appUser.getAppRoles();
        List<String> roles = appRoles.stream().map(r -> r.getRole()).collect(Collectors.toList());
        User.UserBuilder userBuilder = User.withUsername(appUser.getUsername()).password(appUser.getPassword())
        .roles(roles.stream().toArray(String[]::new));
        return userBuilder.build();
    }
}
```

```
@Configuration @EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
@AllArgsConstructor
public class SecurityConfig {
    private UserDetailsServiceImpl userDetailsService;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws Exception {
        httpSecurity.formLogin().loginPage("/login").defaultSuccessUrl("/",true).permitAll();
        ....
        httpSecurity.userDetailsService(userDetailsService);
        return httpSecurity.build();
    }
}
```

UserDetailsService Authentication

```
@Entity
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class AppUser {
    @Id
    private String userId;
    private String username;
    private String password;
    private String email;
    @ManyToMany(fetch = FetchType.EAGER)
    private List<AppRole> appRoles=new ArrayList<>();
}
```

```
@Entity
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class AppRole {
    @Id
    private String role;
}
```

```
public interface AppRoleRepository extends JpaRepository<AppRole,String> {
}
```

```
public interface AppUserRepository extends JpaRepository<AppUser,String> {
    AppUser findByUsername(String username);
}
```

```
public interface AccountService {
    AppUser addUser(String username, String email, String password, String confirmPassword);
    AppRole addRole(String role);
    void addRoleToUser(String username, String role);
    void removeRoleFromUser(String username, String role);
    AppUser loadUser(String username);
}
```

UserDetailsService Authentication

```
@Service @Transactional
@Slf4j @AllArgsConstructor
public class AccountServiceImpl implements AccountService {
    private AppUserRepository appUserRepository;
    private AppRoleRepository appRoleRepository;
    private PasswordEncoder passwordEncoder;
    @Override
    public AppUser addUser(String username, String email, String password, String confirmPassword) {
        if(!password.equals(confirmPassword)) throw new RuntimeException("Password not match");
        AppUser appUser=AppUser.builder()
            .userId(UUID.randomUUID().toString())
            .username(username)
            .email(email)
            .password(passwordEncoder.encode(password))
            .build();
        AppUser savedUser = appUserRepository.save(appUser);
        return savedUser;
    }
}
```

UserDetailsService Authentication

@Override

```
public AppRole addRole(String role) {  
    AppRole appRole=appRoleRepository.findById(role).orElse(null);  
    if (appRole!=null) throw new RuntimeException("Role already exist");  
    appRole=AppRole.builder()  
        .role(role)  
        .build();  
    return appRoleRepository.save(appRole);  
}
```

@Override

```
public void addRoleToUser(String username, String role) {  
    AppUser appUser=appUserRepository.findByUsername(username);  
    AppRole appRole=appRoleRepository.findById(role).get();  
    appUser.getAppRoles().add(appRole);  
    appUserRepository.save(appUser);  
}
```

UserDetailsService Authentication

@Override

```
public void removeRoleFromUser(String username, String role) {  
    AppUser appUser=appUserRepository.findByUsername(username);  
    AppRole appRole=appRoleRepository.findById(role).get();  
    appUser.getAppRoles().remove(appRole);  
    appUserRepository.save(appUser);  
}
```

@Override

```
public AppUser loadUser(String username) {  
    AppUser appUser=appUserRepository.findByUsername(username);  
    if(appUser==null) throw new RuntimeException("User not found");  
    return appUser;  
}  
}
```

UserDetailsService Authentication

@Bean

```
CommandLineRunner start(PatientRepository patientRepository, PasswordEncoder passwordEncoder, AccountService accountService){  
    return args -> {  
        patientRepository.save(new Patient(null, "Mohamed", new Date(), false, 42));  
        patientRepository.save(new Patient(null, "Imane", new Date(), true, 98));  
        patientRepository.save(new Patient(null, "Yassine", new Date(), true, 342));  
        patientRepository.save(new Patient(null, "Laila", new Date(), false, 123));  
  
        accountService.addRole("USER");  
        accountService.addRole("ADMIN");  
        accountService.addUser("user1", "user1@gmail.com", "1234", "1234");  
        accountService.addUser("user2", "user2@gmail.com", "1234", "1234");  
        accountService.addUser("admin", "admin@gmail.com", "1234", "1234");  
  
        accountService.addRoleToUser("user1", "USER");  
        accountService.addRoleToUser("user2", "USER");  
        accountService.addRoleToUser("admin", "USER");  
        accountService.addRoleToUser("admin", "ADMIN");  
    };  
}
```

Table App_Roles

role
ADMIN
USER

Table App_User

user_id	email	password	username
2215a817-a947-4305-99f1-2bc039f72a77	admin@gmail.com	\$2a\$10\$ZsCDe979mODnAa6da9KetuTp8cioUqayK0nFJKB90zR...	admin
24611a31-af7b-4183-a5b5-a823148c1e4e	user1@gmail.com	\$2a\$10\$9eXYB7m.l4Ck8tcOd9WSnu3bKcsnW2lRdjieYOF16SH...	user1
480948ea-3001-499a-8c66-82307885c456	user2@gmail.com	\$2a\$10\$BoYREXAEj3faiQ7dG3TCxeCYtb3jp/WyNxIGtpP3ZGW...	user2

Vues Thymeleaf (Contextualisation)

```
<ul class="dropdown-menu">
  <li th:if="${#authorization.expression('hasRole('ADMIN'))}">
    <a class="dropdown-item" th:href="@{/admin/formPatient}">Nouveau</a></li>
  <li>
    <a class="dropdown-item" th:href="@{/user/index}">Chercher</a>
  </li>
</ul>
```

```
<td th:if="${#authorization.expression('hasRole('ADMIN'))}">
  <a onclick="return confirm('Etes vous sûre?)" class="btn btn-danger"
    th:href="@{/admin/deletePatient(id=${p.id}, keyword=${keyword}, page=${currentPage})}">
    <i class="bi bi-trash"></i> Delete
  </a>
</td>
<td th:if="${#authorization.expression('hasRole('ADMIN'))}">
  <a class="btn btn-danger"
    th:href="@{/admin/editPatient(id=${p.id})}">
    <i class="bi bi-pencil"></i> Edit
  </a>
</td>
```

Formulaire d'authentification

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8"> <title>Login</title>
  <link rel="stylesheet"
href="/webjars/bootstrap/5.2.3/css/bootstrap.min.css">
  <link rel="stylesheet" href="/webjars/bootstrap-
icons/1.10.3/font/bootstrap-icons.css">
</head>
<body>
  <div class="row mt-3">
    <div class="col-md-6 offset-3">
      <div class="card">
        <div class="card-header">Authentication</div>
        <div class="card-body">
          <div th:if="${param.error}">
            <p class="text-
danger">[[${session.SPRING_SECURITY_LAST_EXCEPTION.message}
]]</p>
          </div>
          <div th:if="${param.logout}">
            <p class="text-warning">You have been logged out.</p>
          </div>
          <form>.....</form>
        </div>
      </div>
    </div>
  </div>
</body>
</html>
```

Authentication

Username

admin

Password

....

☐ Remember me

Login

```
<form method="post" th:action="@{/login}">
  <input type="text" class="form-control" id="username" name="username">
  <input type="password" class="form-control" id="password" name="password">
  <input class="form-check-input" type="checkbox" name="remember-me"> Remember me
</form>
```

Formulaire d'authentification

```
<form method="post" th:action="@{/login}">
  <div>
    <div class="mb-3 mt-3">
      <label for="username" class="form-label">Username</label>
      <input type="text" class="form-control" id="username" placeholder="username" name="username">
    </div>
    <div class="mb-3 mt-3">
      <label for="password" class="form-label">Password</label>
      <input type="password" class="form-control" id="password" placeholder="password" name="password">
    </div>
  </div>
  <div class="form-check mb-3">
    <label class="form-check-label">
      <input class="form-check-input" type="checkbox" name="remember-me"> Remember me
    </label>
  </div>
  <button type="submit" class="btn btn-primary">Login</button>
</form>
```

Application Spring Boot

@SpringBootApplication

public class HospitalAppApplication {

public static void main(String[] args) {

SpringApplication.run(HospitalAppApplication.class, args);

}

@Bean

CommandLineRunner start(PatientRepository patientRepository, PasswordEncoder passwordEncoder, AccountService accountService){

return args -> {

patientRepository.save(new Patient(null,"Mohamed",new Date(),false,42));

patientRepository.save(new Patient(null,"Imane",new Date(),true,98));

patientRepository.save(new Patient(null,"Yassine",new Date(),true,342));

patientRepository.save(new Patient(null,"Laila",new Date(),false,123));

accountService.addRole("USER");

accountService.addRole("ADMIN");

accountService.addUser("user1","user1@gmail.com","1234","1234");

accountService.addUser("user2","user2@gmail.com","1234","1234");

accountService.addUser("admin","admin@gmail.com","1234","1234");

accountService.addRoleToUser("user1","USER");

accountService.addRoleToUser("user2","USER");

accountService.addRoleToUser("admin","USER");

accountService.addRoleToUser("admin","ADMIN");

};

}

@Bean

CommandLineRunner commandLineRunner(JdbcUserDetailsManager jdbcUserDetailsManager){

PasswordEncoder passwordEncoder=passwordEncoder();

return args -> {

UserDetails admin=

User.withUsername("admin").password(passwordEncoder.encode("1234")).roles("USER","ADMIN").build();

UserDetails

user1=User.withUsername("user1").password(passwordEncoder.encode("1234")).roles("USER").build();

jdbcUserDetailsManager.createUser(admin);

jdbcUserDetailsManager.createUser(user1);

};

}

@Bean

PasswordEncoder passwordEncoder() {

return new BCryptPasswordEncoder();

}

}

Consulter la session

@RestController

public class MyRestController {

 @GetMapping("/principal")

 public Map<String, Object> principal(Authentication authentication, HttpSession httpSession){

 UsernamePasswordAuthenticationToken authenticationToken = (UsernamePasswordAuthenticationToken) authentication;

 Map<String, Object> map = new HashMap<>();

 Enumeration<String> attributeNames = httpSession.getAttributeNames();

 while (attributeNames.hasMoreElements()){

 String nextElement = attributeNames.nextElement();

 map.put(nextElement, httpSession.getAttribute(nextElement));

 }

 map.put("sessionId", httpSession.getId());

 map.put("creation time", httpSession.getCreationTime());

 map.put("LastAccessedTime", httpSession.getLastAccessedTime());

 map.put("MaxInactiveInterval", httpSession.getMaxInactiveInterval());

 return Map.of("username", authenticationToken.getName(), "authorities", authenticationToken.getAuthorities(), "session", map);

 }

 @GetMapping("/authentication")

 public Map<String, Object> authentication(){

 Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

 return

 Map.of("authentication", authentication, "principal", authentication.getPrincipal(), "authclass", authentication.getClass().getName());

 }

}

```

{
  "username": "admin",
  "authorities": [
    {
      "authority": "ROLE_ADMIN"
    },
    {
      "authority": "ROLE_USER"
    }
  ],
  "session": {
    "org.springframework.security.web.csrf.HttpSessionCsrfTokenRepository.CSRF_TOKEN":
  {
    "token": "c275b9d9-2504-4326-ad2f-fffacf5aa4fd",
    "parameterName": "_csrf",
    "headerName": "X-CSRF-TOKEN"
  },
    "sessionId": "F7D6CC7FB02FB639018162AFFD91D121",
    "creation time": 1680744614352,
    "LastAccessedTime": 1680744616407,
    "SPRING_SECURITY_CONTEXT": {
      "authentication": {
        "authorities": [
          {
            "authority": "ROLE_ADMIN"
          },
          {
            "authority": "ROLE_USER"
          }
        ]
      },
      "details": {
        "remoteAddress": "0:0:0:0:0:0:1",
        "sessionId": "93C676018B2D610CD5C9BDE58AD90812"
      }
    }
  }
}

```

<http://localhost:8085/principal>

Consulter la session

```

"authenticated": true,
"principal": {
  "password": null,
  "username": "admin",
  "authorities": [
    {
      "authority": "ROLE_ADMIN"
    },
    {
      "authority": "ROLE_USER"
    }
  ],
  "accountNonExpired": true,
  "accountNonLocked": true,
  "credentialsNonExpired": true,
  "enabled": true
},
"credentials": null,
"name": "admin"
}
},
"MaxInactiveInterval": 120
}
}

```